

¿Cómo se configuran los *timeouts* en sistemas reales? Un estudio empírico en proyectos Apache

Alejandro Ramos Guillén

Escuela Superior Politécnica del Litoral (ESPOL), Guayaquil, Ecuador
aleramos@espol.edu.ec

Resumen Los *timeouts* son el mecanismo más extendido para tolerar fallos en sistemas distribuidos, y casi todas las instalaciones corren con el valor por defecto que trae el proyecto, porque calibrarlo exige un conocimiento del despliegue que quien lo instala rara vez tiene. Se sabe poco sobre qué valores se están entregando y sobre qué base se eligieron. Siguiendo un protocolo escrito antes de recolectar, se extrajeron 340 *timeouts* de la documentación oficial de 11 proyectos de la Apache Software Foundation, con su valor y unidad originales y su fuente exacta, clasificados en una taxonomía de diez tipos según el evento que dispara la espera. Los valores recorren nueve órdenes de magnitud, de 1 ms a 21 d, con mediana de 30 s, pero se concentran en muy pocas cifras: quince valores redondos cubren el 75 % de los normalizables, encabezados por 1 min y 30 s. Las medianas por tipo progresan de forma coherente con la magnitud física de cada espera. Sobre una muestra de 40 *timeouts* rastreada en incidencias y en la historia de commits, en el 85 % no se localiza justificación alguna del valor y ninguno de los casos restantes se apoya en un experimento. En 44 de los 340 *timeouts* la documentación tampoco alcanza para usarlos sin abrir el código: falta el valor, la unidad o qué significa el cero. Este es un estudio de caracterización: describe lo que los proyectos entregan, no establece qué valor es correcto, y sus cifras deben leerse como referencia y como detector de anomalías, no como recomendación validada.

Keywords: timeouts · configuración · sistemas distribuidos · estudio empírico · Apache

1. Introducción

Ningún sistema distribuido puede distinguir con certeza un componente caído de uno lento. Ante esa imposibilidad, casi todo el software real adopta la misma salida práctica: fijar un plazo y, al agotarse, dar la operación por perdida. El *timeout* es el mecanismo de tolerancia a fallos más extendido de la disciplina y el único que funciona sin cooperación del componente que falla.

Ese plazo no lo elige quien despliega el sistema, sino quien lo escribió. Cada proyecto entrega decenas de *timeouts* con un valor por defecto, y ese valor es el que corre en la inmensa mayoría de las instalaciones, porque calibrarlo exige medir la red, las pausas del recolector de basura y la carga real. Cuando está

mal elegido, las consecuencias son visibles: Dai et al. [2] documentan fallos en cascada por timeouts en sistemas de servidor en la nube, y motivan su estudio con una caída de cinco horas de Amazon DynamoDB en 2015.

Lo que se conoce está del lado de las consecuencias. Existe caracterización empírica de los *bugs* de timeout, pero no del insumo que los origina: qué tipos se configuran, qué números traen y con qué respaldo. Sin ese inventario, quien hereda una configuración no tiene contra qué contrastarla.

En el presente trabajo se construye ese inventario a partir de los valores por defecto tal como los publica la documentación oficial de cada proyecto, conservando la trazabilidad hasta la fuente de la que proviene cada dato. Lo que se cuenta es el timeout y no el fallo, lo que hace que este estudio resulte complementario, y no alternativo, a la literatura existente sobre problemas de configuración temporal.

Como resultado se caracterizan 340 timeouts provenientes de 11 proyectos de la Apache Software Foundation, conservando para cada uno su valor y su unidad originales, el valor normalizado a la unidad canónica y la URL exacta de la fuente de la que fue extraído. Sobre ese conjunto se define una taxonomía operativa de diez tipos, determinada por el evento que dispara la espera, y se caracteriza la distribución de los valores por tipo de timeout y por categoría de proyecto. Todo el procedimiento se rige por un protocolo escrito antes de la recolección, en el que quedaron registrados los criterios de inclusión, las decisiones de alcance adoptadas en los proyectos con configuración dispersa y las desviaciones ocurridas durante el trabajo.

2. Background

Un *timeout* es un límite temporal sobre una operación: agotado el plazo sin que concluya, el sistema la abandona y reintenta, falla o declara muerto a un componente remoto. Elegir su valor es negociar entre falsos positivos, que provocan reintentos y agravan la congestión que los causó, y detección tardía, que retiene hilos y memoria mientras espera. El equilibrio depende del despliegue concreto, que es lo que el autor de la biblioteca desconoce al fijar el valor por defecto.

Agruparlos todos bajo la misma etiqueta impide compararlos, porque cada uno responde a una magnitud física distinta. El de *conexión* limita el establecimiento del canal y se compara con la latencia de red; el de *lectura de socket*, la espera de datos sobre un canal ya abierto; el de *petición* o RPC cubre la operación completa. Los *latidos* (*heartbeats*) miden el intervalo entre señales de vida y la expiración de *sesión* invalida el estado compartido. Se suman los plazos de *lease* y elección de líder, el cierre por *inactividad*, la espera entre reintentos o *backoff* y el apagado ordenado (*graceful shutdown*).

Un timeout puede además traer un valor con significado reservado en lugar de una duración. Por convención, 0 y -1 indican ausencia de límite o mecanismo desactivado, y `null` que el valor se hereda de otro parámetro. Mezclarlos en un promedio distorsiona cualquier conclusión, así que aquí se analizan por separado.

3. Descripción del problema

Todo sistema distribuido decide en algún punto cuánto espera antes de dar una operación por perdida. Esa decisión se materializa en un timeout con un valor por defecto, y ese valor es el que corre en la mayoría de las instalaciones, porque quien despliega rara vez puede calibrarlo. Un valor bajo produce falsos positivos y reintentos en cascada; uno alto retiene recursos y retrasa la detección de fallos reales. Yin et al. [4] mostraron que buena parte de los errores de configuración reportados no son erratas sino valores legales pero inadecuados para el despliegue, que es exactamente la situación de un timeout heredado sin calibrar. Aun así, no se sabe de forma sistemática qué valores se están entregando ni sobre qué base se eligieron.

Este trabajo caracteriza empíricamente esa práctica mediante cinco preguntas de investigación (*research questions*, RQ):

- RQ1** ¿Qué tipos de timeouts se configuran y con qué frecuencia?
- RQ2** ¿Qué valores por defecto se usan y cómo se distribuyen?
- RQ3** ¿Qué justificación del valor es *localizable* en las fuentes públicas del proyecto, y de qué tipo: experimento, heurística, herencia o ninguna razón dada?
- RQ4** ¿Qué tan completa es la documentación del valor por defecto: se publica el valor, se declara su unidad, se explica qué significan los valores especiales?
- RQ5** ¿Qué recomendaciones se derivan para desarrolladores sin experiencia previa en el ajuste de timeouts?

Las cinco se fijaron antes de recolectar. RQ3 se reformuló y la RQ4 original se reemplazó una vez vista la evidencia disponible; ambos cambios constan con su motivo en el protocolo.

4. Metodología

El protocolo se escribió antes de la recolección. Lo que se cuenta es el timeout individual, un ajuste configurable con nombre propio que impone un límite temporal sobre alguna operación, y cada uno ocupa una fila.

Selección del corpus. Los proyectos se eligieron uno a uno, no al azar, buscando cubrir los tipos de sistema donde los timeouts son centrales. Un proyecto entra si es proyecto Apache de primer nivel activo, publica documentación de configuración versionada, pertenece al ecosistema JVM, es de naturaleza distribuida y contribuye a cubrir las categorías consideradas: coordinación, mensajería, bases de datos, procesamiento/almacenamiento, RPC, web e integración. Se planificaron doce proyectos y se recolectaron once: Apache Thrift quedó excluido por no publicar valores por defecto, lo que impide registrar una sola fila sin violar la regla de trazabilidad. Esa baja sesga el corpus, que solo puede contener proyectos que documentan, hacia las buenas prácticas. Nombrar el proyecto que cae y el criterio por el que cae, en lugar de presentar el corpus ya depurado, sigue la práctica de Chávez-Moreno y Abad [1] para estudios de minería de repositorios.

Extracción. Para cada proyecto se fijó su fuente primaria antes de extraer nada, y de ella se tomó, por timeout, el nombre, el valor, la unidad y la descripción tal como aparecen; no se ejecutó ninguno de los sistemas estudiados. Son candidatos los timeouts cuyo nombre contiene `timeout`, `ttl`, `expiry`, `deadline` o `linger`, y los que contienen `wait`, `interval`, `delay` o `period` cuando la descripción confirma semántica temporal; se excluyen contadores de reintentos, booleanos, factores de escala y parámetros de concurrencia. Un timeout que no encaje en ningún patrón queda fuera sin dejar rastro, de modo que el conteo por proyecto se queda corto; en sentido contrario, algunos incluidos están al límite, como los intervalos periódicos o los plazos de caché. Cada fila conserva valor y unidad originales junto al valor normalizado a milisegundos; si la documentación no declara la unidad, la fila se marca y queda fuera del análisis en lugar de recibir una inferida. Ninguna fila existe sin su URL de origen. Los proyectos con configuración dispersa obligaron a decisiones de alcance caso por caso, la más discutible en Camel, del que se tomaron tres componentes de los cientos disponibles y cuyo recuento depende por completo de esa elección.

Clasificación. Los timeouts se clasifican en una taxonomía de diez categorías definidas por el evento que dispara la espera, cada una con definición, ejemplo y regla de desempate frente a sus vecinas. Las categorías son operativas y no canónicas, y su asignación es interpretativa: exige leer la descripción y decidir qué evento dispara la espera, lectura que admite matices en los timeouts que quedan entre dos categorías vecinas.

Justificaciones. Sobre una segunda muestra estratificada de 40 timeouts se rastrea el origen de cada valor en el sistema de incidencias, con búsqueda de frase exacta y lectura de las tres incidencias más antiguas que lo mencionan, y en la historia de commits del repositorio oficial. Cada hallazgo se etiqueta como experimento, heurística, herencia o arbitrario, con la URL correspondiente. La búsqueda tiene un límite de tiempo por timeout; agotado, se registra como no encontrado junto con la constancia de dónde se buscó, porque la proporción sin justificación localizable es en sí misma un resultado.

5. Resultados

5.1. Cobertura

Se recogieron 340 timeouts de la documentación oficial de 11 proyectos de la Apache Software Foundation (Tabla 1). De ellos, 269 (79,1 %) son normalizables a milisegundos y son los únicos que entran en las distribuciones; 59 traen un valor especial (Sección 5.3) y 12 no declaran unidad. El reparto es desigual: Kafka aporta 114 timeouts y Curator uno solo, así que las comparaciones entre proyectos se hacen en proporciones y no en conteos. Al ser un corpus elegido proyecto a proyecto dentro del ecosistema JVM de Apache, los porcentajes que siguen describen estos once y no una población mayor. Nueve nombres de Kafka aparecen repetidos porque el mismo timeout se documenta en varias superficies de configuración; comparten valor, así que las distribuciones apenas lo acusan.

Tabla 1. Cobertura por proyecto. *Normaliz.* son los timeouts con valor por defecto convertible a milisegundos, los únicos que entran en la Figura 1.

Proyecto	Categoría	Total	Normaliz.	Especial	Sin unidad
kafka	Mensajería	114	99	15	0
hdfs	Proc./almacenamiento	72	66	5	1
yarn	Proc./almacenamiento	48	42	5	1
hbase	Bases de datos	23	13	3	7
zookeeper	Coordinación	20	8	12	0
camel	Integración	17	12	4	1
cassandra	Bases de datos	15	14	0	1
tomcat	Web	11	6	4	1
activemq	Mensajería	11	6	5	0
dubbo	RPC	8	3	5	0
curator	Coordinación	1	0	1	0
Total		340	269	59	12

5.2. RQ1: tipos de timeout y su frecuencia

La Tabla 2 da la distribución por categoría. Las tres más frecuentes son REQUEST_RPC, RETRY_BACKOFF y HEARTBEAT_KEEPAIVE. La categoría de descarte OTRO concentra un 42,9 % a la espera de que la clasificación se cierre, por lo que las frecuencias relativas, calculadas sobre una taxonomía propia, son provisionales. Las medianas por tipo no lo son: salen solo de lo ya clasificado.

Tabla 2. Timeouts por tipo. *Filas* cuenta todos los de la categoría y *n* solo los normalizables, que producen la mediana y la banda donde cae la mitad central de los valores. Unidad canónica: milisegundos, expresada en la unidad legible más cercana. La mediana describe lo que estos once proyectos entregan, no lo que se ha demostrado correcto: úsese como punto de partida y como detector de anomalías, nunca como valor validado (Secciones 5.4 y 7). La última columna es la relación esperada con los tipos vecinos y se deriva de la semántica de cada espera, no de los datos.

Tipo	Filas	<i>n</i>	Mediana	Banda	$p_{25}-p_{75}$	Coherencia
RETRY_BACKOFF	29	27	1 s	100 ms–10 s	$\ll$	latido
HEARTBEAT_KEEPAIVE	22	19	5 s	3 s–30 s	$\leq$	1/3 sesión
LEASE_ELECTION	19	12	10 s	1,25 s–52,5 s	$>$	latido
SHUTDOWN	18	11	15 s	250 ms–1 min	$\geq$	petición
CONNECT	18	15	30 s	10 s–1 min	$<$	petición
REQUEST_RPC	38	34	30 s	8,75 s–2 min	$>$	conexión
READ_SOCKET	21	18	45 s	2,5 s–1 min	$<$	petición
SESSION	21	18	55 s	45 s–10 min	$\geq$	3× latido
OTRO	146	108	5 min	10 s–18,75 min	–	
IDLE	8	7	5 min	1 min–9 min	$\gg$	petición

Ordenados por mediana los tipos forman una progresión: 1 s en las esperas entre reintentos, 5 s en los latidos, 10 s en la elección de líder, 15 s en el apagado

ordenado, 30 s en conexión y petición RPC, 45 s en lectura de socket y 55 s en la expiración de sesión. La Figura 1 muestra la dispersión dentro de cada uno.

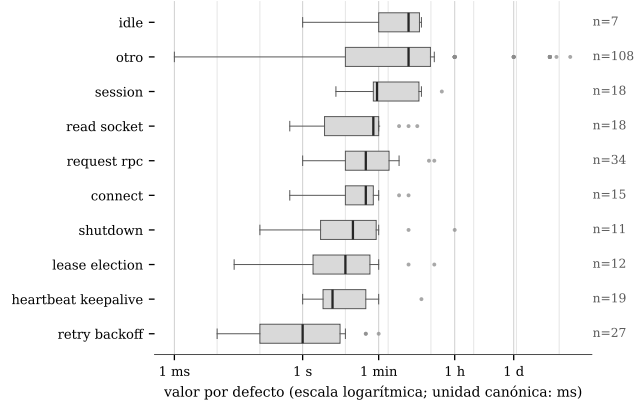


Figura 1. Distribución de los valores por defecto por tipo de timeout. Eje horizontal en escala logarítmica; unidad canónica: milisegundos, rotulada en la unidad legible equivalente. Excluye valores especiales y timeouts sin unidad.

5.3. RQ2: valores por defecto y su distribución

Los 269 valores normalizables recorren nueve órdenes de magnitud, de 1 ms a 21 d, con mediana de 30 s; la mitad central de ellos cae entre 5 s y 5 min. La distribución dista de ser continua: la Figura 2 la muestra concentrada en unas pocas cifras, encabezadas por 1 min (33 timeouts, 12,3 %) y 30 s (24, 8,9 %). Quince cifras redondas habituales¹ cubren 201 de los 269 timeouts (75 %).

Por categoría de proyecto la mediana va de 1 s en coordinación y RPC a 1 min en procesamiento/almacenamiento y bases de datos, con 20 s en integración, 30 s en mensajería y 45 s en web. Las medianas de los tipos con más observaciones son estables; RPC ($n = 3$) y web ($n = 6$) tienen tan pocas que las suyas solo son indicativas.

Valores especiales. Los 59 timeouts con valor especial (17,4 %) quedan fuera de las distribuciones pero son un resultado en sí mismos: 19 traen 0, 8 traen -1 y 8 traen null. De los 28 cuyo valor es cero o negativo, solo 13 explican en la documentación qué significa; en los 15 restantes hay que deducirlo del código o del contexto. Otros 13 no declaran valor por defecto alguno.

5.4. RQ3: justificación localizable del valor

Sobre la muestra estratificada de 40 timeouts se rastreó el origen de cada valor en dos canales: el sistema de incidencias del proyecto, con búsqueda de

¹ 1 s, 2 s, 3 s, 5 s, 10 s, 15 s, 20 s, 30 s, 1 min, 2 min, 5 min, 10 min, 15 min, 30 min y 1 h.

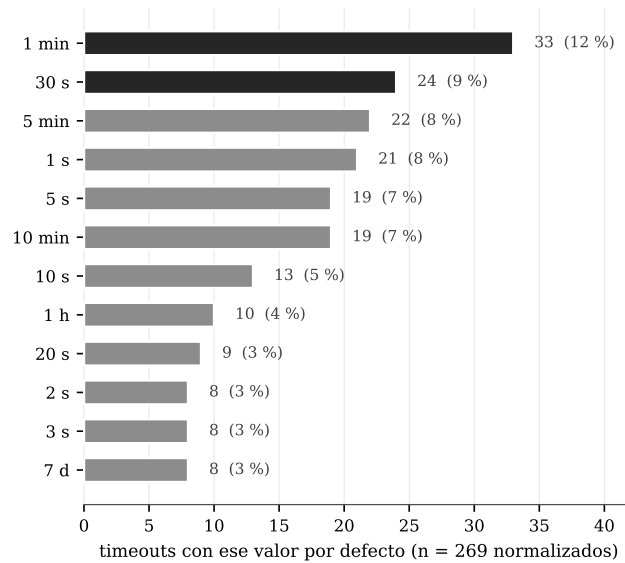


Figura 2. Los doce valores por defecto más frecuentes, sobre los 269 normalizables. El eje horizontal cuenta timeouts; el vertical lista valores de tiempo, no categorías ordinales.

frase exacta y lectura de las tres incidencias más antiguas que mencionan el timeout, y la historia de commits del repositorio oficial. La búsqueda tuvo un tope de tiempo por timeout y todo lo consultado quedó registrado, incluida la ausencia de resultado.

El resultado es que en 34 de los 40 casos (85 %) no se localizó ninguna justificación, y que ninguno de los seis restantes se apoya en un experimento: dos heurísticas, una herencia y tres casos con commit o incidencia sobre el valor pero sin razón dada. Las dos heurísticas son explícitas. En Cassandra, quien sube el umbral de consulta lenta de 200 a 500 ms lo razona así en el hilo: «I know this is very high but I prefer to be on the safe side». En Kafka, la propuesta de diseño que eleva `zookeeper.session.timeout.ms` de 6 a 18s argumenta un cambio de entorno, de centros de datos controlados a nubes inestables, y decide con experiencia operativa: «Based on our experience, genuine failures are rare»; el documento no explica por qué 18 y no otro valor. La herencia es el `socket.soTimeout` de Tomcat, cuyo valor es por definición el de `connectionTimeout`.

Ese 85 % mide localizabilidad, no inexistencia: quedaron sin revisar las listas de correo y el historial línea a línea de los archivos de valores por defecto, de modo que la justificación que existe es esa o más. Lo defendible es que la razón no está al alcance de quien configura el sistema.

5.5. RQ4: completitud de la documentación

Para usar un timeout sin leer el código hacen falta tres cosas: valor publicado, unidad declarada y, si el valor es especial, una explicación de qué significa. De

los 340 timeouts, 296 (87,1 %) cumplen las tres y 44 (12,9 %) fallan en al menos una.

Lo que falta nunca es la prosa, sino lo cuantitativo: los 340 tienen descripción, pero 13 (3,8 %) no publican valor, 25 (7,4 %) no declaran unidad —un 30 puede ser milisegundos o segundos— y de los 28 con cero o negativo solo 13 explican qué implica, dejando 15 donde el lector no distingue entre mecanismo desactivado, espera indefinida y plazo nulo. La completitud varía fuerte por proyecto: Cassandra no tiene timeouts incompletos y Kafka falla en el 4 % de las suyas, frente al 62 % de Dubbo, el 39 % de HBase y el 30 % de ZooKeeper.

6. Discusión

Números redondos. Tres de cada cuatro valores del corpus son una de quince cifras redondas. 1 min aparece 33 veces y 30s otras 24. Una magnitud calibrada no se comporta así. Si el valor por defecto de `request.timeout.ms` saliera de medir latencias en un despliegue real, no habría razón para que cayera en 30s exactos, ni para que otros veintitrés timeouts de proyectos distintos cayeran en el mismo número. Lo que la distribución sugiere es herencia y convención. Alguien eligió un número cómodo, funcionó, y el siguiente proyecto lo copió.

El rastreo de RQ3 refuerza esa lectura: en 34 de 40 casos no hay razón localizable y en ninguno hay un experimento. Que un valor sea convencional no lo vuelve malo; lo que se pierde es la información, porque el proyecto entrega un número sin decir bajo qué supuestos funciona.

El orden de magnitud sí responde a la semántica. Las medianas por tipo forman una escalera que coincide con lo que cada espera representa físicamente: un *backoff* se mide contra el tiempo que tarda en despejarse una sobrecarga momentánea; una sesión, contra la pausa más larga que puede sufrir el proceso remoto sin estar muerto. Kafka documenta esa relación: con `session.timeout.ms` en 45s y `heartbeat.interval.ms` en 3s, advierte que el segundo no debe pasar de un tercio del primero. Quien invierta esa relación tendrá un cliente que se declara muerto entre latido y latido.

Lo que la documentación calla. Los resultados de RQ3 y RQ4 apuntan al mismo problema desde dos lados. La razón del número casi nunca está al alcance del lector, y en 44 timeouts falta además el dato duro: valor, unidad o el significado del cero. Un caso concreto, `observer.reconnectDelayMs` de ZooKeeper, trae cero sin una palabra sobre qué implica. El extremo son dos proyectos que estaban planificados: Thrift no publica valores y quedó fuera del corpus, y Curator declara explícitamente uno solo. Los dos son de uso masivo.

7. Recomendaciones

Para quien configura sin experiencia previa. Quien tiene que fijar un timeout necesita un número, no una advertencia. La Tabla 2 lo da por tipo: la mediana

como punto de partida y la banda donde cae la mitad central de lo observado. El procedimiento tiene tres pasos. Primero, partir de la mediana del tipo de espera y no del nombre del timeout, porque el nombre engaña: el `connectionTimeout` de Tomcat mide la espera de datos sobre una conexión ya aceptada, no su establecimiento. Segundo, comprobar la coherencia con los plazos vecinos usando la última columna de esa tabla: la espera entre reintentos por debajo del latido, el latido por debajo de un tercio de la sesión, la conexión por debajo de la petición completa. Eso detecta la inversión de dos plazos relacionados, y hace falta vigilarla porque dentro del corpus la razón entre sesión y latido va de 1,7 a 600 según el proyecto. Tercero, ajustar con una medición propia: el plazo de una petición debería cubrir con margen la latencia que en el despliegue real solo supera una de cada cien. Y antes de dejar un 0 o un número negativo tal como viene, conviene averiguar qué significa, porque más de la mitad de esos casos no lo declara y un timeout desactivado por accidente no produce error, simplemente se queda esperando.

Para quien escribe la documentación de un proyecto. Se recomienda publicar el valor por defecto junto con su unidad y la razón de la elección. Escribir «30s, elegido para tolerar dos latidos perdidos» cuesta poco y convierte un número opaco en una decisión comprensible. Los 25 timeouts sin unidad declarada y los 15 ceros sin explicar son los casos donde ese costo no se pagó, y obligan al lector a recurrir al código.

Para quien diseña sistemas distribuidos. Si el valor adecuado depende del despliegue y quien despliega no está en condiciones de calcularlo, cabe preguntarse si debe fijarse de antemano. Los detectores de fallos acumulativos [3] sustituyen el umbral fijo por una estimación construida sobre la latencia observada, y los dos enfoques ya conviven en el propio corpus: Cassandra expone un `phi_convict_threshold` para su detector y a la vez entrega plazos fijos en el resto de su configuración. Esos mecanismos no se evalúan aquí, pero 201 de 269 valores redondos y 34 de 40 sin razón localizable son un argumento cuantitativo para considerarlos.

8. Trabajo futuro

Quedan dos frentes abiertos, en orden de cercanía.

El primero es agotar los canales que faltan para RQ3. El rastreo del origen de los valores cubrió el sistema de incidencias y los mensajes de commit, pero no las listas de correo ni el historial línea a línea de los archivos de valores por defecto, que es donde suele estar el cambio que fijó cada número. Cerrar esos dos canales permitiría saber cuánto del 85 % sin justificación localizable corresponde a justificación inexistente y cuánto a justificación mal indexada.

El segundo es extender el protocolo fuera del ecosistema JVM, donde las convenciones de nombres y unidades son distintas, y aplicarlo a familias de sistemas más recientes, en particular a los servidores del Model Context Protocol, donde los timeouts cumplen el mismo papel y las convenciones aún se están formando.

9. Conclusiones

Tabla 3. Resumen de hallazgos por pregunta de investigación, sobre los 269 timeouts normalizables de un total de 340. Unidad canónica: milisegundos.

RQ	Hallazgo principal
RQ1	Predominan petición/RPC (38), <i>backoff</i> (29) y latidos (22); las medianas progresan de 1 s a 55 s según la magnitud de la espera.
RQ2	Mediana de 30 s sobre nueve órdenes de magnitud; quince cifras redondas cubren el 75 % y un 17,4 % trae un valor especial.
RQ3	En 34 de 40 (85 %) no se localiza justificación; de los 6 restantes, ninguno es un experimento (2 heurísticas, 1 herencia, 3 sin razón).
RQ4	296 de 340 (87 %) publican valor, unidad y significado; fallan 44 (13 sin valor, 25 sin unidad, 15 ceros sin explicar).
RQ5	Cuatro recomendaciones derivadas de RQ1–RQ4 (Sección 6).

La Tabla 3 resume los hallazgos: los valores se concentran en cifras redondas y casi nunca hay una razón localizable detrás, aunque el orden de magnitud sí responde a la semántica de cada espera. De ese contraste sale la contribución práctica del trabajo, los valores de referencia por tipo de la Sección 7, que describen el consenso del ecosistema sin presentarlo como verdad validada.

Referencias

1. Chávez-Moreno, Á.C., Abad, C.L.: Openlambdaverse: A dataset and analysis of open-source serverless applications (Aug 2025). <https://doi.org/10.48550/arXiv.2508.01492>
2. Dai, T., He, J., Gu, X., Lu, S.: Understanding real-world timeout problems in cloud server systems. In: 2018 IEEE International Conference on Cloud Engineering (IC2E). pp. 1–11. IEEE Computer Society, Orlando, FL, USA (Apr 2018). <https://doi.org/10.1109/IC2E.2018.00022>
3. Hayashibara, N., Défago, X., Yared, R., Katayama, T.: The ϕ accrual failure detector. In: 23rd International Symposium on Reliable Distributed Systems (SRDS). pp. 66–78. IEEE Computer Society, Florianópolis, Brazil (Oct 2004). <https://doi.org/10.1109/RELDIS.2004.1353004>
4. Yin, Z., Ma, X., Zheng, J., Zhou, Y., Bairavasundaram, L.N., Pasupathy, S.: An empirical study on configuration errors in commercial and open source systems. In: Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP). pp. 159–172. ACM, Cascais, Portugal (Oct 2011). <https://doi.org/10.1145/2043556.2043572>